

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

Linux sur SoC FPGA

Du registre au driver, de la chaîne Yocto aux flux de données

Mathieu Léonardon
IMT Atlantique · Formation du 18 novembre 2026

Objectifs de la journée

En sortant, vous savez :

1. **rendre une IP de la PL utilisable depuis Linux** : `/dev/mem`, UIO ou driver noyau, lequel choisir, et comment diagnostiquer quand ça ne marche pas
2. **lire votre propre chaîne**, du `.xsa` à l'invite Linux : qui fait quoi, donc où intervenir dans un projet Yocto
3. **choisir une architecture pour des flux de données** entre le FPGA et le processeur : registres, ou AXI-Stream et DMA

Le matin sur la carte, l'après-midi en cours, démonstrations et discussion.

Déroulé

Horaire	Bloc	Forme
10 h 00	Objectifs, positionnement	
10 h 10	Marche 1 : `/dev/mem`	cours, TP 1
11 h 10	Marche 2 : device tree et driver (déjeuner à 12 h 30)	cours, TP 2
14 h 00	La chaîne complète ; Yocto et l'EDF	cours, démonstration
14 h 45	Marche 3 : AXI-Stream et DMA (pause à 15 h 30)	cours, démo
15 h 45	Analog Devices : IIO et `libiio`	discussion
16 h 20	Bilan	

Périmètre

Aujourd'hui

- La frontière entre le matériel et le logiciel, vue depuis Linux
- Zedboard (Zynq-7000) : TP sur la chaîne SEIML (noyau 5.4), démonstrations sur l'EDF (noyau Linux 6.18)

Hors périmètre

- Conception d'IP, chemin critique, horloges : volet architecture et HLS
- Bibliothèques Analog Devices : on en donne la clé de lecture (IIO, `libiio`), pas le détail
- Temps réel

Wooclap de positionnement

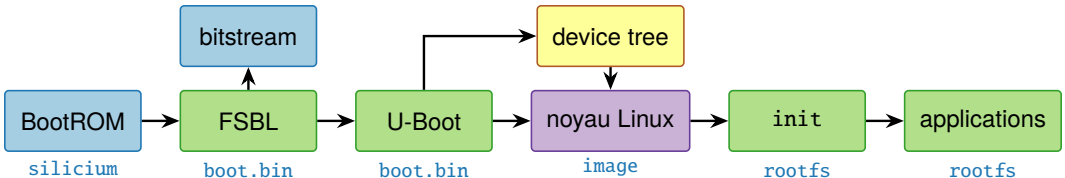
Cinq questions pour savoir d'où l'on part.

`wooclap.com` code à l'écran

1

Marche 1 : espace utilisateur

La chaîne de démarrage



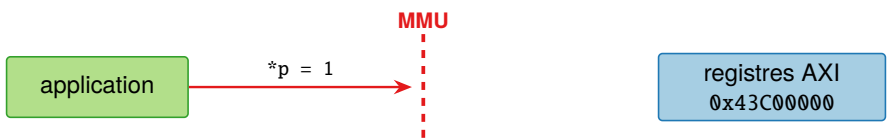
- **BootROM**, puis **FSBL** : configure le PS, charge le bitstream, puis U-Boot
- **U-Boot** : charge le noyau et son device tree, lui passe ses options (`bootargs`)
- **Noyau** : démarre ses drivers d'après le device tree, monte sa racine (ici par NFS, depuis le poste), lance `init`

> La carte est déjà démarrée. Le détail de la chaîne, et qui la construit : cet après-midi.

Sources : `github.com/Xilinx/u-boot-xlnx` (U-Boot), `github.com/Xilinx/linux-xlnx` (noyau)

Problème : la mémoire d'un programme est virtuelle

- En *bare metal*, écrire à l'adresse `0x43C00000` atteint le registre
- Sous Linux, chaque programme a son propre espace d'adresses, traduit par la **MMU** ; seul le noyau en fixe la table
- Une adresse que le noyau n'a pas donnée au programme : erreur de segmentation



/dev/mem et mmap (1/2)

```c
#define CHENILLARD_BASEADDR 0x43C00000

/* 1. ouvrir la mémoire physique : man 4 mem */
int fd = open("/dev/mem", O_RDWR | O_SYNC);

/* 2. en projeter une page :      man 2 mmap */
volatile uint32_t *reg = mmap(NULL, 4096,
    PROT_READ | PROT_WRITE, MAP_SHARED,
    fd, CHENILLARD_BASEADDR);

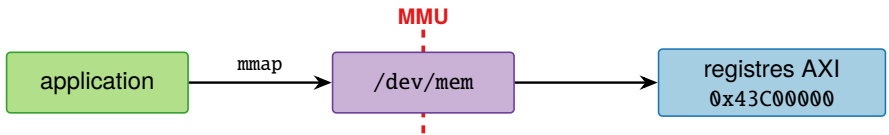
/* 3. lire et écrire les registres */
reg[0] = 1;      /* mode    */
reg[1] = 7;      /* vitesse */

/* 4. défaire : munmap, close */
```

- `/dev/mem` : un fichier spécial qui *est* la mémoire physique
- `mmap` : « noyau, projette cette page physique dans mon espace virtuel »
- L'adresse doit être un multiple de la taille de page

/dev/mem et mmap (2/2)

- `reg` est une nouvelle adresse virtuelle, par exemple `0xb6f45000`, que la MMU envoie vers `0x43C00000`
- Toujours en **mode utilisateur** : le noyau a seulement ajouté la page du chenillard à la table du programme



TP 1 : piloter le chenillard par `/dev/mem`

TP **40 min. Carte prête : chaîne SEIML, noyau 5.4, racine NFS sur le poste**

1. Lire `/proc/cmdline` ; compiler pour l'ARM, lancer sur la carte
2. Écrire dans les registres du chenillard avec `devmem`, sans rien programmer
3. Écrire le programme C : `open`, `mmap`, mode et vitesse en arguments
4. Pour les rapides : lire les interrupteurs, et faire suivre la vitesse du chenillard

Sujet : **bonben.github.io/partage/tp**

2

Marche 2 : device tree et driver

Bilan de la marche 1 (démonstration : interrupteurs, deux programmes)

Ce qui marche

- Quelques lignes, aucun code noyau
- Idéal pour vérifier qu'un bitstream et ses adresses sont bons

Ce qui ne va pas

- Réservé à root, et accès à *toute* la mémoire : une adresse fausse et l'on écrit ailleurs
- Aucune protection entre programmes : deux programmes, un chenillard
- L'adresse physique est dans le programme : nouveau design, nouvelle compilation
- **Pas d'interruption** : on ne peut que scruter le registre en boucle

Le device tree

Pourquoi

Un même noyau pour des milliers de cartes : les adresses ne peuvent pas être compilées dedans. U-Boot donne au noyau une description du matériel.

La partie PL est générée depuis le `.xsa` ; ce que l'outil ne peut pas deviner (votre IP, votre driver) s'ajoute à la main.

```
btns_gpio: gpio@41200000 {
  compatible = "imt,axi-btns";
  reg = <0x41200000 0x10000>;
  interrupts = <0 29 4>;
};
```

`reg` : où sont les registres, et sur quelle taille.

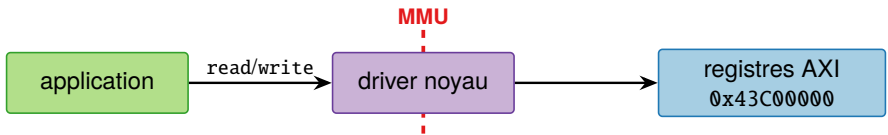
`interrupts` : quelle ligne (ici la SPI 29 du GIC, sur niveau haut).

`compatible` : le noyau cherche un **driver** qui annonce la même chaîne.

Trois niveaux pour atteindre une IP

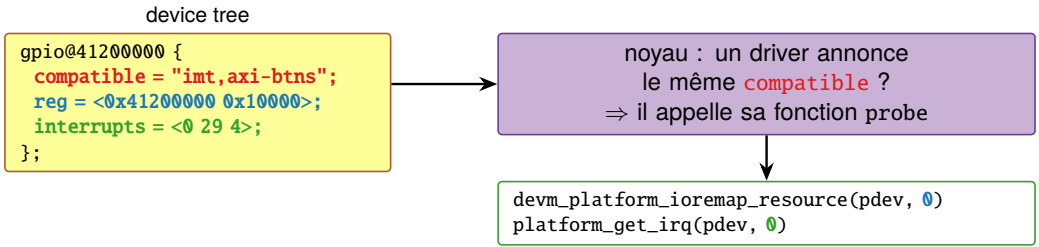
	`/dev/mem`	UIO	Driver noyau
Code noyau	aucun	aucun (générique)	le vôtre
Adresse	programme	device tree	device tree
Interruptions	non	lecture bloquante	dans le noyau
Droits	root, tout ou rien	par fichier	par fichier
Interface	registres bruts	registres bruts	`read`, `write`, `ioctl`
Usage	mise au point	IP simple	produit, débit

Un driver : du code noyau propriétaire d'un périphérique



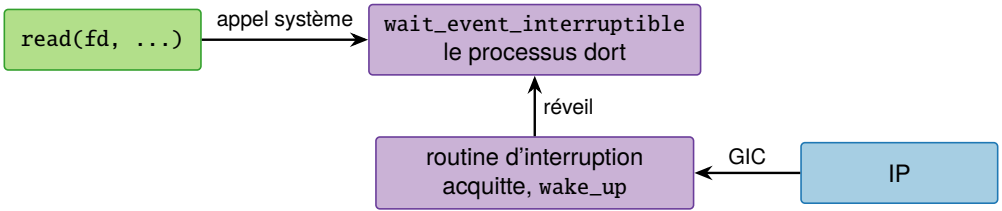
- Un **module** (`.ko`) chargé dans le noyau : `insmod`, `rmmod`, `lsmod`
- Il réserve les registres, les projette (`ioremap`), prend l'interruption
- Il expose un fichier, par exemple `/dev/chenillard` : l'application fait `open`, `read`, `write`, sans connaître ni adresse ni registre

Driver de plateforme : du `compatible` à `probe`



- Le driver **demande** son adresse et son interruption : rien n'est compilé dedans
- L'appariement a lieu au chargement du module, ou à l'arrivée du nœud (overlay)

Interruption et lecture bloquante



- L'application ne scrute pas : elle dort dans `read`, sans consommer de CPU
- La routine d'interruption reste courte : acquitter le matériel, noter l'événement, réveiller
- C'est le schéma de presque tous les drivers d'acquisition

Diagnostiquer : où regarder

Question	Où regarder
Le bitstream est-il chargé ?	`/sys/class/fpga_manager/fpga0/state`
Le noyau connaît-il mon IP ?	`/sys/bus/platform/devices`
`probe` a-t-elle été appelée ?	`dmesg`, `/sys/bus/platform/drivers`
Qui a réservé ces registres ?	`/proc/iomem`
L'interruption arrive-t-elle ?	`/proc/interrupts`, avant et après
L'IP répond-elle sans driver ?	`devmem`

> `probe` jamais appelée : souvent un `compatible` qui diffère d'un caractère.

TP 2 : lire, casser, modifier un driver

TP 1 h. Fournis : chenillard (adresse en dur), boutons (driver de plateforme)

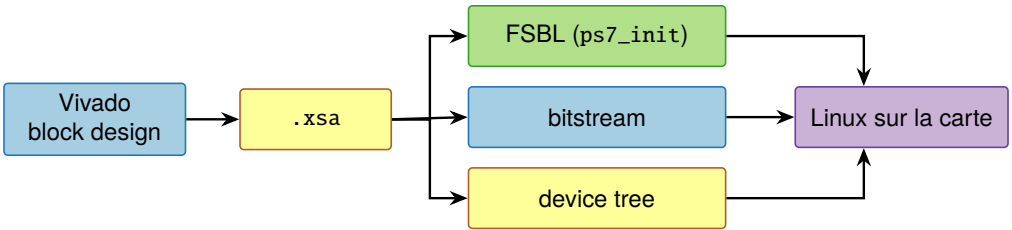
1. Lire le driver du chenillard ; compiler, charger, piloter ; `dmesg`, `/proc/iomem`
2. **Panne** : les modules se chargent, mais `/dev/btns` ne marche pas. Trouver pourquoi, corriger
3. **Modification** : `hexdump /dev/chenillard` n'affiche rien ; corriger `read`
4. Pour les rapides, puis en démonstration : interruption et lecture bloquante

Sujet : **bonben.github.io/partage/tp**

3

La chaîne complète : Yocto et IÉDF

Du `.xsa` à l'invite Linux



- Le `.xsa` est le **seul lien** entre le projet Vivado et le logiciel : configuration du PS, bitstream, adresses et interruptions des IP
- Tout le reste (U-Boot, noyau, système de fichiers) est du logiciel libre générique, configuré pour la carte

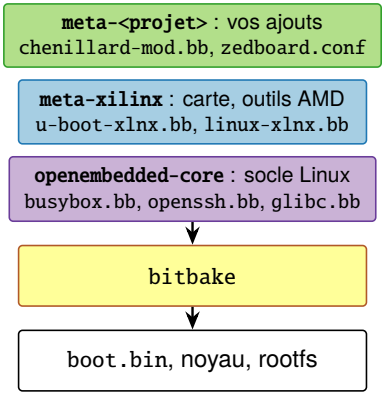
Qui fait quoi, et d'où ça vient

	Dépend de votre matériel par	Produit par Yocto
FSBL	la configuration du PS (`.xsa`)	dans `boot.bin`
Bitstream	le design de la PL (`.xsa`)	dans `boot.bin`, ou chargé depuis Linux (`.bit.bin`)
U-Boot	son device tree, sa configuration	dans `boot.bin`
Noyau	sa configuration	`uImage`
Device tree	le `.xsa`, plus vos ajouts	`system.dtb`
Rootfs	vos applications, vos modules	image `.wic` (carte SD), ou archive (NFS)

Yocto : vocabulaire

Yocto construit tout depuis les sources, en suivant des recettes ; l'outil s'appelle `bitbake`.

- **Recette** (`.bb`) : comment récupérer, compiler et installer *un* logiciel
- **Couche** (`meta-*`) : un dossier de recettes par thème (socle, AMD, projet)
- **Machine** (`.conf`) : la carte visée (processeur, noyau, device tree)
- **Image** (`.bb`) : la liste des paquets mis dans le rootfs



Une recette : un module noyau

```
SUMMARY = "Driver du chenillard"
LICENSE = "GPL-2.0-only"
inherit module
SRC_URI = "file://Makefile \
           file://chenillard.c"
```

```
$ bitbake chenillard-mod
$ bitbake edf-linux-disk-image
```

- Une recette est un fichier `.bb` dans votre couche (ici `chenillard-mod.bb`)
- `inherit module` : la classe sait compiler contre le noyau de la machine
- `SRC_URI` : des fichiers locaux, ou un dépôt git et une révision
- Règle : on ne modifie jamais les couches d'AMD ; tout ajout va dans la vôtre

Comment tout s'emboîte dans votre couche

```
meta-seiml/
  conf/machine/zedboard-seiml.conf
  recipes-kernel/chenillard-mod/
    chenillard-mod.bb
    files/Makefile, chenillard.c
  recipes-core/images/
    edf-linux-disk-image.bbappend
```

```
# conf/local.conf : choisir la machine
MACHINE = "zedboard-seiml"

# edf-linux-disk-image.bbappend : ajouter au
    rootfs
IMAGE_INSTALL:append = " chenillard-mod
    axi-btns-mod"
```

- **Machine** (`zedboard-seiml.conf`) : indique l'architecture ARM, le noyau (`linux-xlnx`) et le device tree
- **Recette** (`.bb`) : définit le paquet `chenillard-mod`
- **Surcouche** (`.bbappend`) : complète une recette d'une autre couche sans la modifier (ici l'image d'AMD, ou le device tree)

L'EDF, le Yocto d'AMD

- AMD a publié avec Vivado 2025.1 l'**EDF** (Embedded Development Framework) : du Yocto sans surcouche
- Il remplace **PetaLinux**, annoncé en voie de retrait : AMD recommande l'EDF pour tout nouveau projet, et fournit un guide de migration. PetaLinux était déjà du Yocto : `meta-user` est une couche ordinaire
- Pour une carte à vous, le point d'entrée reste le `.xsa`

Ce que Yocto apporte

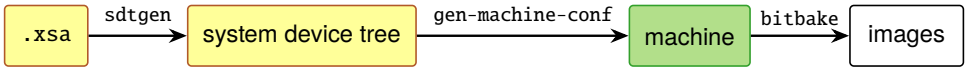
Des outils standards ; un projet réduit à des couches sous git, donc reproductible.

Ce que ça coûte

Une première construction longue (heures, dizaines de Go) ; un vocabulaire à apprendre.

Guide de migration PetaLinux vers EDF : https://edf.docs.amd.com/en/latest/petalinux-to-edf-migration-guide.html

Du `.xsa` à la machine Yocto



- **System device tree** : la description de tout le matériel (PS, PL, tous les processeurs) ; le device tree de Linux et la configuration du FSBL en sont tirés
- `gen-machine-conf` écrit le fichier de machine dans votre projet (pour la démonstration : `zedboard-seiml`)
- À refaire quand le block design change la configuration du PS, les adresses ou les interruptions

Où intervenir : correspondance avec PetaLinux

Vous voulez	PetaLinux	EDF (Yocto)
prendre en compte un nouveau design	`petalinux-config --get-hw-description`	`sdtgen`, puis `gen-machine-conf`
activer un driver du noyau	`petalinux-config -c kernel`	`bitbake -c menuconfig virtual/kernel`
décrire votre IP, ajouter au rootfs	`project-spec/meta-user`	votre couche : `.dtsi` ou overlay, `IMAGE_INSTALL:append`
ajouter un module noyau	`petalinux-create -t modules`	recette avec `inherit module`
construire	`petalinux-build`	`bitbake <image>`

Démonstration : une Zedboard sous EDF

Sur la carte

- Machine Yocto créée depuis le `.xsa` du TP
- U-Boot 2026.01, noyau 6.18
- Démarrage en QSPI, racine par NFS
- Drivers : une recette chacun, dans notre couche

```
$ ssh zed
# uname -r
# cat /proc/device-tree/model
# cat /sys/class/fpga_manager/fpga0/state
# lsmod
# grep -e chen -e btns /proc/iomem
# cat /proc/interrupts
```

Charger le bitstream depuis Linux

```
/dts-v1/; /plugin/;
&fpga_full {
  firmware-name = "seiml.bit.bin";
  chenillard@43c00000 {
    compatible = "imt,chenillard";
    reg = <0x43c00000 0x10000>;
  };
};
```

```
# D=/sys/kernel/config/device-tree/overlays
# mkdir $D/seiml
# cat seiml.dtbo > $D/seiml/dtbo
```

- **FPGA manager** : le noyau programme la PL
- **Overlay** : un morceau de device tree ajouté en marche ; il nomme le bitstream *et* décrit ce qu'il contient
- Les drivers correspondants sont appelés aussitôt
- Intérêt : changer de design sans refaire `boot.bin`

4

Marche 3 : flux de données

Problème

Un registre convient à une commande. Pas à un flux d'échantillons.

- **AXI-Lite** : une transaction complète par mot de 32 bits, et c'est le processeur qui fait chaque transfert
- Depuis une application : un appel système par lecture, ou une boucle de scrutation
- Un convertisseur produit à cadence fixe, que le logiciel soit prêt ou non ; sous Linux, un processus peut être suspendu plusieurs millisecondes

Il faut que le matériel écrive lui-même les données en mémoire, et prévienne le logiciel quand un bloc est prêt.

Deux chemins : commandes et données



- **Commandes et état** : registres AXI-Lite, par le port GP du PS (le processeur est maître)
- **Données** : AXI-Stream jusqu'à un DMA, qui écrit en DDR par un port HP du PS (la PL est maître)
- Le processeur n'intervient qu'une fois par bloc : sur l'interruption de fin de transfert

AXI-Stream

Quatre signaux suffisent

- `tdata` : la donnée
- `tvalid` : la source a une donnée
- `tready` : le récepteur peut la prendre
- `tlast` : dernière donnée du paquet

Transfert quand `tvalid` et `tready` sont à 1 sur le même front.

- Pas d'adresse : un flux, dans un seul sens
- `tready` à 0 : le récepteur **retient** la source (contre-pression)
- Une IP de calcul peut attendre ; un convertisseur ne le peut pas

Si la source ne peut pas attendre, il faut une FIFO, et savoir quoi faire quand elle est pleine.

AXI DMA : du flux vers la mémoire

Registre	Adr.	Rôle
`S2MM_DMACR`	`0x30`	marche, remise à zéro, interruptions
`S2MM_DMASR`	`0x34`	état, erreurs, acquittement
`S2MM_DA`	`0x48`	adresse **physique** du tampon
`S2MM_LENGTH`	`0x58`	taille en octets ; l'écrire lance le transfert

1. Le logiciel donne une adresse et une taille
2. Le DMA écrit le flux en DDR, par rafales, jusqu'à `tlast`
3. Interruption ; `LENGTH` relu donne le nombre d'octets reçus

> Canal S2MM (flux vers mémoire), mode simple. Le canal MM2S fait l'inverse. Référence : PG021.

Pourquoi il faut du code noyau

Le DMA parle en adresses physiques

- Un `malloc` donne des pages virtuelles, éparpillées en mémoire physique, déplaçables
- Le DMA veut un tampon **contigu**, qui ne bouge pas
- Le processeur a un cache : il doit relire la mémoire, pas son cache

Ce que fournit le noyau

- `dma_alloc_coherent` : tampon contigu, avec son adresse pour le DMA
- La réserve CMA, dimensionnée au démarrage (`cma=64M`)
- `mmap` du tampon vers l'application : aucune copie

Par `/dev/mem` : possible, fragile, et sans interruption.

Démonstration : le design et le driver

```
/* read() de /dev/capture : un paquet */
iowrite32(RS | IOC_IRQ, dma + S2MM_DMACR);
iowrite32(c->buf_dma,   dma + S2MM_DA);
iowrite32(BUF_SIZE,     dma + S2MM_LENGTH);

wait_event_interruptible_timeout(
    c->wq, c->done, HZ);

n = ioread32(dma + S2MM_LENGTH);
copy_to_user(ubuf, c->buf, n);
```

- Source AXI-Stream : un signal triangulaire et un **numéro d'échantillon**, paquets de 4096 mots
- AXI DMA sur HP0, un registre de commande
- Driver `axis_capture` : 200 lignes, les mêmes briques que ce matin (`compatible`, `probe`, interruption)

Démonstration : deux sources, deux comportements

Mode flux : la source attend

Elle n'avance que si le DMA est prêt (`tready`). Débit maximal, numérotation continue d'un paquet au suivant.

```
# ./capture flux 1000
```

Mode CAN : la source n'attend pas

Un échantillon tous les *N* cycles, DMA prêt ou non. Entre deux paquets, le temps que le logiciel relance le DMA, des échantillons sont perdus.

```
# ./capture can 1000 16 99
```

Relancer le DMA depuis le logiciel à chaque bloc ne suffit pas pour une acquisition continue.

Acquisition continue : ce qu'il faut ajouter

- Une **FIFO** entre la source et le DMA, pour absorber les pauses courtes
- Le mode **scatter-gather** du DMA : une liste de tampons préparée d'avance, le matériel passe de l'un au suivant sans le logiciel ; en mode **cyclique**, la liste reboucle
- Dans le noyau, l'interface **dmaengine** : le driver AMD `xilinx_dma` gère l'AXI DMA, votre driver demande un canal et soumet des transferts, sans toucher aux registres
- Un indicateur de **débordement** lisible par le logiciel : savoir qu'on a perdu vaut mieux que l'ignorer

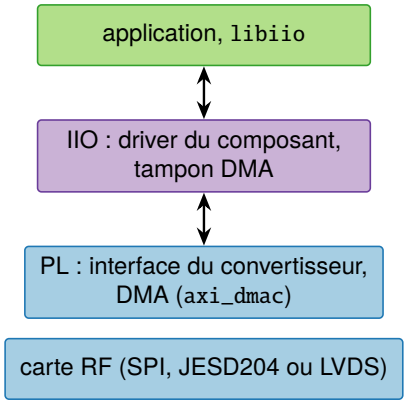
> Ordre de grandeur : un flux de 32 bits à 100 MHz, c'est 400 Mo/s au plus sur un port HP. La limite vient ensuite de ce que le logiciel fait des données.

5

Analog Devices : IIO et `libiio`

IIO et `libiio` : clé de lecture d'une chaîne Analog Devices

- **IIO** (Industrial I/O) : le sous-système du noyau pour convertisseurs et capteurs. Chaque composant a ses *canaux* et ses *attributs* (fréquence, gain) dans `/sys/bus/iio/devices`
- Les données passent par un **tampon IIO**, alimenté par DMA : le schéma de la démonstration, en version continue
- **`libiio`** : la bibliothèque en espace utilisateur, en local ou par le réseau (`iiod`) ; `iio_info`, `iio_readdev`, `pyadi-iio`



Analog Devices sous Yocto : `meta-adi`

- Une couche Yocto ordinaire (`meta-adi-xilinx`) ; PetaLinux ne fait que l'ajouter
- Elle apporte le **noyau ADI** (AD9361, `axi_dmac`, IIO), les **device trees** de référence, `libiio`
- Une branche par version d'outils : `2023_R2` pour 2023.2, `main` pour 2025.1

Attention avec l'EDF

- **Pas de support officiel de l'EDF par ADI** (seul PetaLinux est validé)
- `main` : même Yocto (Scarthgap) que l'EDF 2025.1 ; noyau ADI et `libiio` réutilisables
- **Device tree** : la recette de `meta-adi` suppose PetaLinux et ne marche pas en l'état (prendre le `.dts` du noyau ADI)

Accéder à une carte à distance

```
$ ssh root@carte
$ scp root@carte:/tmp/rx.bin .
$ iio_info -u ip:carte
$ iio_readdev -u ip:carte \
    -s 4096 <capture> > rx.bin
```

- VPN pour le réseau ; `ssh` et `scp` pour lancer et rapatrier
- `iiod` sur la carte : `libiio` et `pyadi-iio` depuis le PC, rien à compiler pour la carte
- Bien pour des captures, pas pour un flux continu

Porter un traitement : les étapes

Étape	Produit	Ce qui peut coincer
Référence	traitement sur PC, données de test	pas de données de référence
Arithmétique	virgule fixe, écart mesuré	précision perdue, vue tard
Partition	PL ou ARM, débits entre les deux	débit PS-PL, mémoire
IP	HDL ou HLS, simulée	ressources, fréquence
Intégration	device tree, driver, application	l'objet de cette journée
Validation	carte contre référence	écarts durs à localiser

Signes qu'un traitement sera difficile à embarquer

Dans la PL

- Calcul flottant indispensable, grande dynamique
- Accès aléatoires à de gros volumes (hors BRAM, tout passe par la DDR)
- Algorithme qui change souvent, branches nombreuses

Sur l'ARM

- Dépendances lourdes (Python scientifique, GPU), mémoire de plusieurs Go
- Débit de données de l'ordre du Go/s à traiter par logiciel
- Latence exigée de l'ordre de la microseconde

Réflexe : chiffrer les débits et la mémoire dès le prototype PC.

Bilan

À retenir

1. **Une IP dans Linux** : `/dev/mem` pour vérifier, UIO pour une IP simple, un driver de plateforme pour un produit. Le device tree fait le lien, par `compatible`, `reg` et `interrupts`
2. **La chaîne** : le `.xsa` porte le matériel ; FSBL, bitstream et device tree en découlent. Yocto construit le tout à partir de couches ; votre projet tient dans la vôtre, et l'EDF est le Yocto d'AMD
3. **Les données** : commandes par registres, flux par AXI-Stream et DMA. Le tampon contigu et l'interruption demandent du code noyau ; l'acquisition continue demande scatter-gather ou IIO

Pour aller plus loin

Drivers	*Linux Device Drivers*, 3e édition (libre, https://lwn.net/Kernel/LDD3) ; supports de formation Bootlin (https://bootlin.com/docs)
Zynq	UG585 (Zynq-7000), UG1085 (Zynq UltraScale+) : manuels de référence
DMA	PG021, *AXI DMA Product Guide* ; dans le noyau, `Documentation/driver-api/dmaengine`
Yocto, EDF	https://docs.yoctoproject.org ; formation Yocto de Bootlin ; documentation de l'EDF et migration PetaLinux (https://edf.docs.amd.com)
Analog Devices	https://wiki.analog.com : IIO, `libiio` ; https://github.com/analogdevicesinc/meta-adi

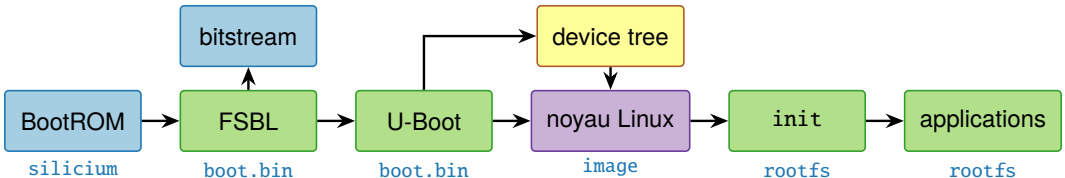
Wooclap de sortie

Les mêmes questions que ce matin, et votre avis sur la journée.

`wooclap.com` code à l'écran

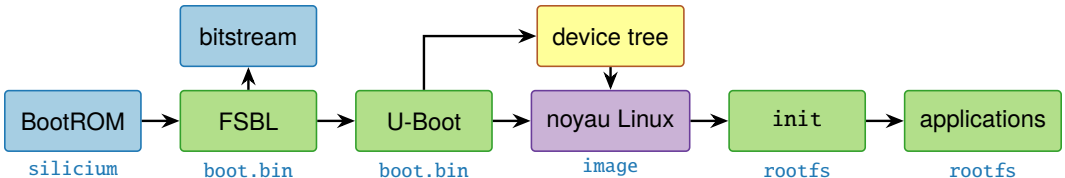
Annexes

Chaîne de démarrage (1/2)



- **BootROM** : dans le silicium ; lit les broches de mode, charge le FSBL depuis la flash ou la carte SD
- **FSBL** : configure horloges, DDR et broches du PS (`ps7_init`, issu du `.xsa`), charge le bitstream, puis l'étage suivant
- **U-Boot** : charge le noyau et son device tree, lui passe ses options (`bootargs`)

Chaîne de démarrage (2/2)



- **Noyau** : démarre ses drivers d'après le device tree, monte le système de fichiers racine, lance `init`
- **Système de fichiers racine** (rootfs) : `init`, bibliothèques, outils, vos applications et vos modules

UIO : le niveau intermédiaire

```
btns@41200000 {
  compatible = "generic-uio";
  reg = <0x41200000 0x10000>;
  interrupts = <0 29 4>;
};
```

```
int fd = open("/dev/uio0", O_RDWR);
volatile uint32_t *reg = mmap(NULL, 0x10000,
    PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
uint32_t n = 1;
write(fd, &n, 4);  /* armer l'interruption */
read(fd, &n, 4);   /* dort en l'attendant */
```

- Driver générique du noyau, `uio_pdrv_genirq` : registres projetés, interruption rendue par une lecture bloquante
- L'acquittement dans l'IP reste à votre charge

> Option du noyau : `uio_pdrv_genirq.of_id=generic-uio`

Lire une fonction probe

```
static int btns_probe(struct platform_device *pdev)
{
  /* allocation de b ; contrôles d'erreur omis */
  b->regs =
      devm_platform_ioremap_resource(pdev, 0);
  b->irq = platform_get_irq(pdev, 0);
  devm_request_irq(&pdev->dev, b->irq, btns_isr,
                   0, "axi_btns", b);
  iowrite32(1, b->regs + GPIO_IER);
  return misc_register(&b->misc);  /* /dev/btns */
}
```

1. les registres : `reg` du device tree, réservés et projetés

2. l'interruption : `interrupts` du device tree

3. le matériel est mis en état

4. le fichier apparaît

`devm_` : libéré au retrait du driver.